

Deep Monkey Studio: An AI-Native Platform for Programmable 3D Worlds

Wenpeng Zhang
Independent Researcher
15184552744@163.com

Technical report — 9 October 2026

Code: <https://github.com/fatasia/DeepMonkey-Studio>

Abstract

AI-assisted 3D creation requires addressable objects with data, behavior, and runtime state, together with feedback from execution. We present Deep Monkey Studio, an AI-native platform combining a modular engine, programmable scenes, and tool-driven agents. The platform connects engineering and 3D models, live data, 2D/3D editing, scripts, plugins, domain computation, physical simulation, and cross-platform delivery, making a 3D world an environment that can be constructed and operated over time. Deep Engine provides WebGPU rendering, programmable materials, render-graph scheduling, a shared resource budget, and browser/native execution interfaces. Asset, scene, and execution contracts connect these capabilities to editors and agents; agents operate through a tool gateway, revision checks, receipts, and persistent checkpoints. We articulate the design motivation from Vibe Coding to Vibe World and use industrial scenes to explain the integrated workflow. Recorded results from 86 targeted tests and engine comparisons characterize selected software behaviors and operating costs. A 1,000-object fixture records seven draw calls while exposing startup and dynamic-update costs. We discuss the environment interfaces this architecture can provide for AI for Science and world-model research, distinguishing available mechanisms from research capabilities still to be evaluated.

1 Introduction

Generating code from natural language can reduce the effort of building software. Constructing an executable 3D world additionally requires geometry, state, behavior, and external data to work together. Model files must become addressable objects, scripts must reference those objects, simulation and domain services must produce inspectable state, and edits must reach a delivery runtime. Code generation is one step in this path.

We describe this motivation as *from Vibe Coding to Vibe World*: expanding AI-assisted creation from code to 3D environments that remain editable, executable, and observable. Vibe World names the authoring vision; its implementation consists of concrete scene representations, engine interfaces, and tool-execution mechanisms.

Deep Monkey Studio uses the programmable 3D scene as its organizing abstraction. Assets supply geometry and properties, bindings connect external state, scripts and domain actions express behavior, and physics sessions provide supported dynamics. Users author scenes through 2D/3D editors, scripts, and plugins. AI participates through project context, ontology targets, and tool

interfaces. Deep Engine supplies resources, materials, graphics execution, and runtime delivery. The integrated application and standalone SDK paths reuse these modules.

We use AI-native to describe addressable objects, contextual information, and callable actions as platform interfaces. An agent can inspect task-scoped information, propose tool operations, receive execution results, and continue from a checkpoint. This is an architectural property rather than a claim of successful world generation from arbitrary prompts.

The report makes three contributions. First, it presents and implements a platform organization connecting authoring and runtime use through asset, scene, and execution contracts. Second, it details Deep Engine’s rendering, residency, programmable materials, and cross-runtime interfaces alongside the complete authoring path formed by editors, model/protocol ingestion, scripts, and plugins. Third, it explains how tool-driven agents and world-execution interfaces participate in these paths, using existing tests and engine measurements to characterize implementation scope and cost. Industrial digital twins serve as a concrete application; AI for Science and world-model research are further uses of the relevant interfaces.

Research questions. RQ1: Which representations and interfaces make a 3D scene editable, programmable, and accessible to AI? RQ2: How do the engine, data connections, and authoring tools divide responsibility and support delivery? RQ3: Which capability and performance conclusions are supported by the execution mechanisms, tests, and measurements? We address these questions through implementation analysis and component measurements.

2 Related Work

Unity treats a game engine as a configurable platform for intelligent-agent research, emphasizing abstractions, interfaces, and enabled research [8]. Our focus is a programmable scene shared across human authoring, AI tool use, domain computation, and delivery, with engineering-model and device integration as concrete workloads. The platform-level contribution concerns reusable organization and execution capabilities; rendering algorithms are one part of that design.

Holodeck studies language-guided generation of embodied environments through assets and spatial constraints [1]. SceneCraft combines structured scene generation with procedural tools [2], while RoboGen connects generated tasks and environments to robotic skill learning [3]. Code2Games connects scene and gameplay representations with engine and behavior feedback [4]. These works inform our use of explicit representations and executable evaluation.

Existing digital-twin software also extends beyond visualization demos. Eclipse Ditto provides device-state middleware [5]; iTwin.js provides infrastructure-twin development libraries [6]; OpenTwins combines 3D visualization, IoT streams, and machine-learning predictions [7]. We compare architectural roles through input fidelity, programmable operations, state ownership, and delivery dependencies. The available results do not provide a common-task ranking of these platforms.

3 Platform Architecture

3.1 Representing a Programmable World

We use world to denote an authoring unit formed by a scene and its associated runtime services. Objects have identity, geometry, properties, transforms, and asset provenance. Bindings connect them to data; scripts and domain actions provide operable behavior; physics sessions and domain

models supply the corresponding state evolution. This representation spans multiple contracts rather than a single scene file or global solver.

The representation supplies common targets for users and AI. Renderers consume drawable resources, data services resolve bindings, action services check targets and preconditions, and editors expose modifiable properties. Engine modules can be used independently, while the complete platform organizes these operation surfaces within projects.

3.2 Three Interface Families and State Ownership

Figure 1 summarizes the system paths. Asset interfaces describe what an input is, where it originates, and what information survives conversion. Scene interfaces describe object composition, binding, and modification. Execution interfaces describe the resources and configuration required by a runtime. These families summarize the implementation without requiring a single global database or transaction.

Let $A = (i, r, p, g, m)$ denote a conceptual asset record: package identity i , revision r , supported profile p , runtime geometry g , and retained hierarchy, properties, and provenance m . Scene objects store asset references, transforms, overrides, and bindings. Reimport produces an asset revision; the scene adapter determines affected references and applicable retained overrides. Rendering-resource identities and canonical ontology targets are consumed through their respective adapters. Similar identifier names do not imply identical lifecycles.

Scene writes, agent checkpoints, physics sessions, and publication candidates have distinct state owners. Cross-layer operations carry target identity, revision, or content hashes for checks at the receiving boundary. This permits independent evolution but does not turn local rollback and receipts into distributed exactly-once execution.

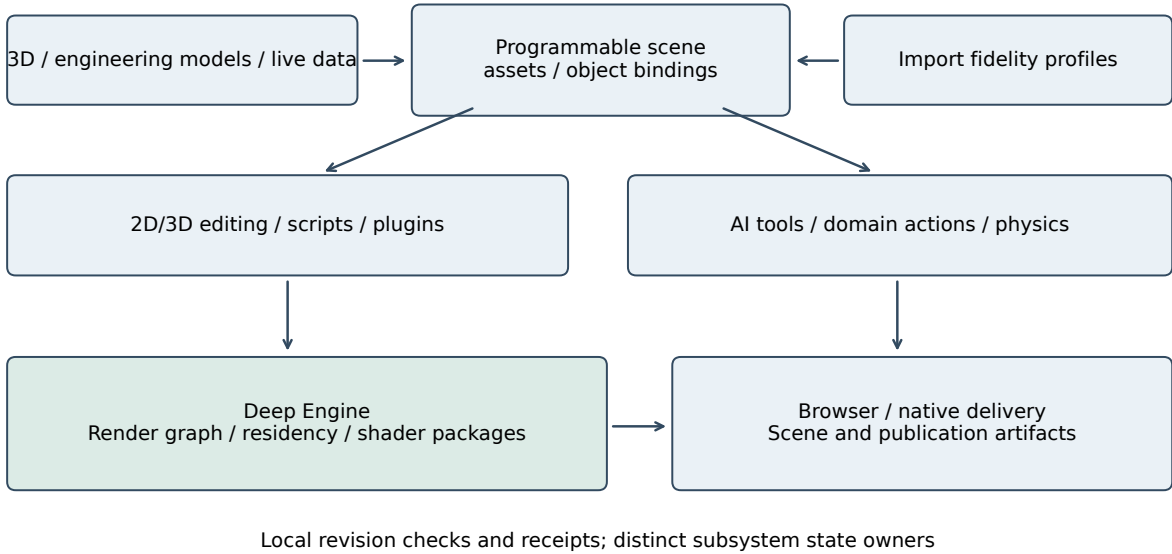


Figure 1: Input, authoring, and execution paths in Deep Monkey. Arrows represent interface connections; subsystems retain distinct state ownership.

Layer	Responsibilities and interfaces
Inputs	Engineering files, device protocols, database queries, event streams; source identity and format profile.
Asset and data	Revised model packages, hierarchy and properties, normalized data rows and events, object bindings.
Authoring	2D/3D editors, dashboards, scripts, plugins, scene transactions, human and agent tools.
Execution	WebGPU rendering, ontology actions, persistent agent runs, headless physics, domain services.
Delivery	Compiled scene artifacts, runtime packages, browser/native consumers, content hashes and runtime evidence.

The implementation combines TypeScript/React application code with Rust/native components. Its existing Three.js viewer and Deep WebGPU backend coexist. Engine-neutral identities and versioned packages connect authoring to execution; each adapter has an explicit supported profile.

3.3 Deep Engine: Rendering, Resources, and Cross-Runtime Execution

Deep Engine provides resource packets, instancing, render scheduling, shader packages, and WebGPU execution. The rendering implementation includes GGX materials, image-based lighting, directional shadows, linear HDR, tone mapping, transparency, and postprocessing. Its glTF path handles supported texture channels, node animation, skinning, and morph targets. Unsupported encoding features produce structured errors.

Resource residency shares a geometry/texture budget. Visible and prefetched chunks request levels of detail; candidates are validated before publication at a frame boundary. Cancellation and superseded requests govern pending uploads and cleanup. Shader packages carry entry points, bindings, vertex layouts, and pass configuration, with shared TypeScript/Rust representations. A versioned 2D display list carries paths, text, images, clipping, and hit regions into native chart rendering.

These mechanisms support local delivery and backend evolution. Product-wide visual equivalence, device-loss recovery, and complete animation/physics coverage require separate acceptance runs from isolated rendering benchmarks.

Scheduling and resource reuse. The render graph validates dependencies and orders passes topologically, producing temporary-resource lifetimes and physical-slot plans. Aliasing requires compatible format, dimensions, sample count, and usage; external resources are excluded from temporary slots. This makes reuse an explicit scheduling decision. For streamed scenes, residency planning retains the coarsest drawable level for visible objects before adding geometry detail and texture mips within the budget. Ownership and cleanup rules handle superseded uploads.

Programmable materials and execution interfaces. The DeepSL surface compilation path translates Standard/Unlit material properties into typed intermediate representations and WGSL. Versioned shader packages specify entry points, bindings, vertex streams, attachments, and render state for validation and pipeline creation by browser and Rust executors. Caches distinguish persistent content from device-specific objects. Scene state is separated from the backend, while the host controls drawing and backend handover. These interfaces serve editor integration, standalone SDK use, and native playback paths.

3.4 Authoring Coverage and Extensibility

The platform organizes 2D pages, 3D scenes, data bindings, scripts, and plugins within a project. Model hierarchy and properties support object discovery and selection; device data supplies bound-object state; dashboards present project data; domain services provide computation for actions and analysis. The engine supplies visual and runtime execution, while editors provide interaction. Scene transactions and publication artifacts connect editing to deployment. This separation supports both modular engine use and an integrated authoring application.

Functional coverage is organized around a task path: ingest engineering models and device data, bind objects, edit scenes and pages, execute domain actions, save, and deliver. Format and protocol breadth determine input coverage; scripts, plugins, and SDKs determine extension points; runtime packages and local deployment determine delivery options. The following sections describe the inspected input paths, operations, and measured scope.

3.5 Model Ingestion and Fidelity

Imports retain source provenance and produce runtime geometry with associated hierarchy and properties. Format coverage is described by version/profile and retained information. Table 1 summarizes the inspected implementation paths and documentation; it is not an exhaustive compatibility certification.

Table 1: Engineering model paths and their scope.

Format	Implemented path or tested scope
glTF / GLB	Runtime exchange, optimization and publication; loader-specific extension coverage.
IFC	BIM ingestion through fragments, preserving a source representation and semantic access.
STEP / STP	Open-source tessellation into runtime geometry, with hierarchy/property sidecars.
FBX; DXF	Scene/animation loading and 2D drawing ingestion through their respective adapters.
OpenUSD	USDA/USDC/USDZ loading with inspected sample coverage; complex composition remains a separate validation target.
JT	Built-in LOD0 triangle geometry for inspected 9.5/10.3 profiles, assembly instances and partial properties/materials; PMI structure is distinct from semantic annotations.
X_T	Built-in text parsing for controlled geometric subsets. Trim, surface-family, naming, color, and assembly losses depend on the profile.

The broader catalog also records candidate formats. A catalog entry is not evidence of a working importer. Native proprietary formats require an admitted built-in/open-source profile before inclusion in the operational coverage matrix. Evaluation must score geometry, assembly structure, properties, interaction, and delivery independently.

3.6 Industrial Protocols and Data Sources

The API dispatches real connector adapters and normalizes returned data for consumers. Continuous subscriptions and bounded previews serve different operational needs.

Interface family	Inspected operations
MQTT; OPC UA	Persistent subscriptions and preview reads. Shared subscription lifecycle; OPC UA quality, timestamps, and configured security handling.
Modbus TCP; BACnet; S7; EtherNet/IP	Bounded reads of registers/coils, properties, PLC addresses, or tags.
HTTP; WebSocket; Kafka; AMQP	Request or bounded message sampling through source-specific adapters.
SNMP; CoAP; TCP; UDP; serial	Source-specific sampling adapters.
SQL and analytical stores	Query adapters including PostgreSQL, MySQL-compatible stores, SQL Server, Oracle, ClickHouse, and time-series sources.
Document and file sources	MongoDB, Elasticsearch, CSV, and Excel adapters.

Persistent MQTT and OPC UA sources feed the event infrastructure. Simulation-result export includes MQTT and SQL paths. Actual device coverage depends on transport settings, address mapping, data types, and vendor behavior; a field evaluation should record these alongside reconnection and event-ordering outcomes.

3.7 Asset Revision and Controlled Editing

Source hashes and package identities associate a revision with its asset. The reimport coordinator prepares updates before publication and supports rollback, generation checks, and conflict handling. The product adapter updates scene snapshots while retaining identity relationships and applicable overrides.

Scene transactions carry a base revision and validated commands. The transaction layer rejects stale revisions and invokes rollback after partial execution, errors, or cancellation. Editor and agent consumers use this path; the API/browser bridge preserves transaction identity and caches outcomes. This gives concurrent work a concrete admission boundary.

3.8 Ontology, Agents, and Simulation

Ontology actions bind canonical targets to published definitions, arguments, scope, and preconditions. Execution produces a receipt. Property constraints have report-only and strict behavior, so the selected mode belongs in each experiment.

The connected industrial agent runtime uses a tool gateway, persisted checkpoints, cancellation, and revision-aware recovery. Its Web workspace exposes resume and cancel operations. A separate task-graph helper supports bounded correction but has no identified product consumer in this audit.

The World API provides reset, step, observe, snapshot, restore, and close operations over a Rapier-based headless runtime. The inspected profile runs at 60 Hz, with at most 256 objects and 600 ticks per step. Observations include poses, contacts, and events; camera/depth/lidar sampling

is outside this profile. Rigid visual models require supported collider declarations. Restore checks snapshot integrity and replaces a session only after successful reconstruction.

4 AI-Native Authoring and Research Interfaces

4.1 From Intent to an Inspectable State Change

The connected agent runtime assembles a decision provider, tool gateway, persisted checkpoints, execution guards, and a memory pipeline. Project and ontology context expose entities that tools can address. A scene-edit operation produces a command plan tied to a base revision; ontology actions additionally carry canonical targets, argument checks, scope, and preconditions. Execution returns results or rejection reasons for the next decision. Cancellation and revision-aware recovery preserve explicit task boundaries.

This path connects language-model decisions to ordinary platform operations. Generated code remains useful for scripts and plugins, while tool calls provide structured operations over existing scenes and services. The backend remains responsible for admission and execution; a model-generated instruction alone does not change scene state. The available tests cover selected tool, transaction, and recovery behaviors, not an end-to-end score for autonomous world creation.

4.2 Complete Authoring and Delivery Paths

A representative industrial task starts with importing an assembly under a supported format profile, identifying components through hierarchy and properties, and binding selected objects to device data. An author adds a dashboard, script, or domain action; an agent can query the resulting context or propose an edit through the same operation surfaces. Supported bodies can enter a physics session. Saving and publication produce artifacts consumed by browser or native paths. This sequence illustrates how the implemented subsystems compose; a paired benchmark of the complete sequence remains future work.

The breadth of the platform comes from covering the stages of this task rather than treating rendering, data, or AI as the entire application. Fidelity profiles describe what survives import; protocol operations describe how state arrives; scripts, plugins, and SDKs describe how behavior is extended; runtime contracts describe how the result is executed and delivered.

4.3 AI for Science and World-Model Research

For AI for Science, scenes can supply an interactive context for measured data, simulation state, and domain-model outputs. The battery service is an implemented example of domain inference within the platform; the scientific validity of each model depends on its own data and evaluation. The platform provides integration and inspection, not universal physical validity.

For world-model research, the relevant substrate is an environment with explicit actions, observations, reset, stepping, and snapshot restoration. The current World API exposes poses, contacts, and events within its bounded rigid-body profile. These interfaces can support collection of transitions and comparison of predicted outcomes with simulator outcomes. Training learned dynamics, visual observation generation, and large-scale benchmark datasets are research extensions rather than demonstrated results in this report.

The distinction matters operationally: the platform supplies environments and interfaces for learning about worlds; it is not itself a pretrained predictive world model. Source access, modular SDKs, and local execution support inspection and customization under the repository license.

5 Evaluation

5.1 Targeted Software Verification

The 9 October 2026 verification run passed 86 tests across nine files: 40 world-runtime tests, 37 API tests covering ontology actions, agent stop/recovery, the World API and editor transaction bridge, five scene-transaction tests, and four task-graph tests. These tests exercise selected admission, replay, ownership, rollback, and recovery behavior. The source review also traced the corresponding product consumers.

5.2 Recorded Rendering Comparison

The 7 October benchmark used three cold contexts per configuration on an i9-12900HX and RTX 4060 Laptop GPU, at 1440×900, DPR 1, with PBR, ACES, and 1024-resolution shadows. Table 2 selects the 1,000-object, 614,294-triangle fixture. Frame P95 is the median across runs of browser RAF measurements; GPU P95 uses timestamp measurements. Deep WebGPU combines a low

Backend	Workload	First frame ms	Frame P95 ms	GPU P95 ms	Draws
Three WebGL	Static	72.7	7.1	4.2	1055
Three WebGPU	Static	283.2	7.1	1.6	1056
Babylon WebGPU	Static	876.0	15.0	4.5	2001
Deep WebGPU	Static	1623.3	7.1	3.1	7
Three WebGL	Dynamic	70.4	7.1	4.1	1055
Deep WebGPU	Dynamic	1733.7	14.0	3.1	7

Table 2: Archived isolated rendering measurements. Draw counters are engine-specific and not fully equivalent.

reported draw count with competitive static frame pacing in this fixture. Its first frame is substantially later, and dynamic frame pacing is worse than the Three WebGL reference. These results identify startup and update costs for further optimization. They do not resolve full-product visual parity or backend promotion.

5.3 Interpretation

The 86 tests support the admission, rollback, replay, and recovery behaviors they exercise. Rendering results expose distinct startup, steady-state, and dynamic-load costs: fewer recorded draw calls do not produce a uniform frame-time advantage in this fixture. We therefore report functional coverage, correctness checks, and performance separately. Quantifying whole-platform benefit requires paired tasks with fixed assets and device inputs, spanning model updates, concurrent editing, interruption recovery, and cross-runtime delivery.

6 Discussion and Limitations

The platform’s practical value lies in editable source, local runtime paths, common identities, and inspection of state changes across tools. These properties make customization and maintenance possible within the applicable license. Commercial readiness additionally depends on sustained field workloads, upgrade behavior, operator usability, and support costs; those measurements are not yet available in this study.

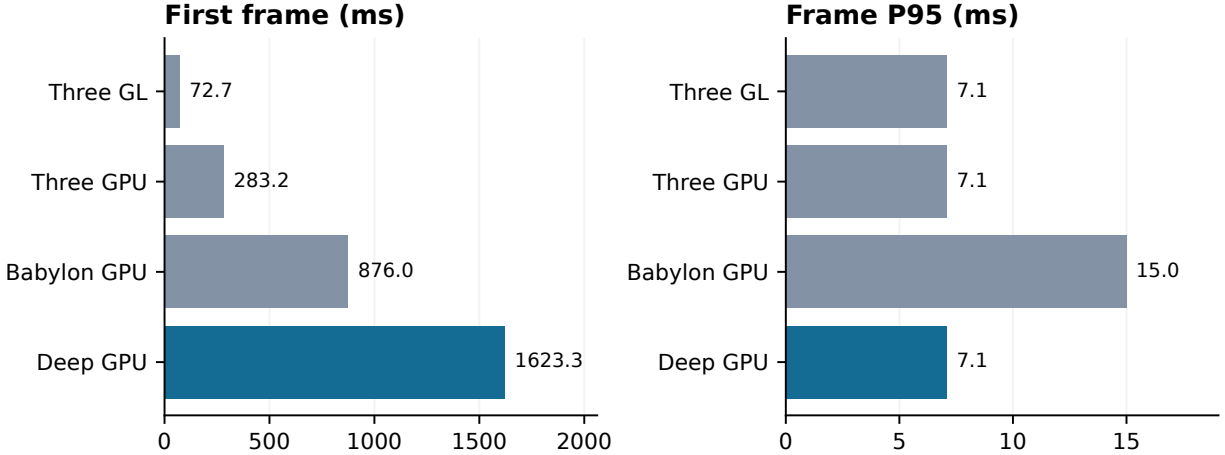


Figure 2: Static 1,000-object fixture from the archived comparison. Startup and steady-state frame pacing expose different costs; lower is better.

Current evidence covers selected software behaviors and isolated rendering. Sequential receipt replay does not establish concurrent exactly-once execution, and transaction atomicity depends on the driver. Format fixtures do not cover all versions or geometries. A complete lifecycle comparison is the main remaining empirical task.

7 Reproducibility and Availability

The platform source is available at <https://github.com/fatasia/DeepMonkey-Studio>. Commit [8bbfd90535db](#) identifies the implementation inspected for this update; historical rendering measurements retain their own recorded revisions. The [Deep Engine documentation](#) describes engine modules and supported profiles, and the [deployment guide](#) provides startup instructions.

The accompanying evidence notes identify source files, reviewed artifacts, test commands, and remaining experiments. Rendering records use their own recorded revision and hardware; manuscript tables retain that distinction from the current implementation audit. A reproducible release must preserve asset bytes, profile declarations, dependency/checkpoint versions, action traces, and expected outcomes.

Deep Monkey Studio is source-available under a modified MIT license with additional corporate conduct and human-rights provisions, named the Deep Monkey Community Source License 1.0 (DMCSL-1.0). The repository license specifies the applicable terms. Article and third-party asset licenses are separate.

Generative AI assisted source inspection, organization, and manuscript drafting.

8 Conclusion

Deep Monkey Studio makes AI-native authoring concrete through addressable scenes, context, tool actions, and execution feedback, with Deep Engine supporting graphics, resources, and cross-runtime use. Model and data ingestion, 2D/3D editors, scripts, plugins, agents, domain services, and delivery form an authoring path for programmable 3D worlds. The motivation from Vibe Coding to Vibe World is thus expressed through inspectable representations and interfaces that extend

beyond one-shot code generation. Industrial applications illustrate this organization, while AI for Science and world-model research can reuse the relevant environment interfaces. Existing tests and measurements characterize selected correctness properties and costs; complete-task efficiency and learning outcomes require independent evaluation.

References

- [1] Y. Yang et al. Holodeck: Language Guided Generation of 3D Embodied AI Environments. CVPR, 2024. <https://arxiv.org/abs/2312.09067>.
- [2] Z. Hu et al. SceneCraft: An LLM Agent for Synthesizing 3D Scenes as Blender Code. ICML, 2024. <https://proceedings.mlr.press/v235/hu24g.html>.
- [3] Y. Wang et al. RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation. ICML, 2024. <https://proceedings.mlr.press/v235/wang24cc.html>.
- [4] W. Wu et al. Code2Games: Enabling Coding Agents for Gaming World Generation. arXiv:2610.05033, 2026. <https://arxiv.org/abs/2610.05033>.
- [5] Eclipse Foundation. Eclipse Ditto: Overview. Accessed 9 October 2026. <https://eclipse.dev/ditto/intro-overview.html>.
- [6] iTwin.js. Getting Started. Accessed 9 October 2026. <https://www.itwinjs.org/learning/>.
- [7] J. Robles, C. Martín, and M. Díaz. OpenTwins: An open-source framework for the design, development and integration of effective 3D-IoT-AI-powered digital twins. arXiv:2301.05560, 2023. <https://arxiv.org/abs/2301.05560>.
- [8] A. Juliani et al. Unity: A General Platform for Intelligent Agents. arXiv:1809.02627. <https://arxiv.org/abs/1809.02627>.